CP468 Assignment 2 — Connect Four AI

Team Setup & Run Guide

This document is for anyone on the team, even with zero prior context on this project. Follow it top to bottom and you'll be able to run everything and understand what it does.

0. What This Project Is

We built three AI agents that play Connect Four (Random, Rule-Based, and Minimax with a heuristic), a shared game engine they all plug into, and an experiment script that plays 30 games between each pair of agents and records win rates and decision times. All of the code and the Report.pdf are already written and tested. What's left:

- Someone runs the experiments once on their own machine and records the real results + machine specs.
- Someone records a 3-5 minute demo video showing the program running.
- Someone zips the final folder and submits it.

These can be split across different people or all done by one person — see Section 6.

1. Getting the Code (GitHub)

The code and report already live in a private GitHub repository:

```
https://github.com/asadmx/cp468-a2-connect4
```

1.1 If you haven't been added yet

Asad needs to add you as a collaborator before you can access it. Ask him to add your GitHub username or the email tied to your GitHub account. You'll get an email invite — accept it before continuing.

If you don't have a GitHub account yet, sign up first at github.com/signup, then send Asad the username you picked.

1.2 Install Git (one-time, if you don't have it)

Check if you already have it:

```
git --version
```

If you get an error, install Git:

- Windows: download from git-scm.com/downloads, run the installer, keep all default options.
- Mac: run "git --version" in Terminal — macOS will usually prompt you to install the Xcode Command Line Tools automatically, which includes git. Click Install.

Close and reopen your terminal after installing before continuing.

1.3 Clone the repository

Windows (PowerShell):

```
cd $env:USERPROFILE cd Downloads git clone https://github.com/asadmx/cp468-a2-connect4.git cd cp468-a2-connect4
```

Mac (Terminal):

```
cd ~/Downloads git clone https://github.com/asadmx/cp468-a2-connect4.git cd cp468-a2-connect4
```

The first time you clone, it will likely open a browser window asking you to log into GitHub and authorize — sign in and it will continue automatically.

Run "dir" (Windows) or "ls" (Mac) — you should see README.md, Report.pdf, and a src folder containing engine.py, agents.py, experiment.py, and main.py.

You now have the exact same code Asad has. Everything from here works the same regardless of how you got the code.

2. One-Time Setup (per machine)

2.1 Check if Python is installed

Open a terminal:

- Windows: press the Windows key, type "PowerShell", press Enter.
- Mac: press Cmd+Space, type "Terminal", press Enter.

Then run:

```
python --version
```

If you see a version number 3.10 or higher, you're set — skip to Section 3. If you get an error, or the version is older, install Python:

- Windows: go to python.org/downloads, download the installer, and during installation CHECK THE BOX that says "Add Python to PATH" before clicking Install.
- Mac: go to python.org/downloads and run the installer, or if you have Homebrew installed run: brew install python3

On Mac, you may need to use python3 instead of python in every command below — try python first, and if it says "command not found", use python3.

3. Running the Code

3.1 Navigate to the code

If you cloned via GitHub (Section 1), you're likely already in the right folder. Otherwise:

Windows (PowerShell):

```
cd $env:USERPROFILE\Downloads\cp468-a2-connect4\src
```

Mac (Terminal):

```
cd ~/Downloads/cp468-a2-connect4/src
```

Run "dir" (Windows) or "ls" (Mac) — you should see engine.py, agents.py, experiment.py, and main.py listed.

3.2 Sanity check — watch a demo match

```
python main.py demo
```

This prints a Connect Four board to the terminal, move by move, as the Rule-Based agent plays against the Minimax agent. If this runs without errors and ends with a result (WIN or DRAW), everything is working correctly.

3.3 Run the real experiments

```
python experiment.py
```

This takes 30-60 seconds. It plays all 3 required agent match-ups (30 games each) and creates two new files in the same folder: results.json and results.csv. These are the official numbers that go in the report — screenshot or copy the console output when it finishes.

3.4 Get your machine specs

These need to go in the report, since decision-time numbers are meaningless without knowing what machine they were measured on. Windows (PowerShell):

```
python --version Get-CimInstance Win32_Processor | Select-Object Name Get-
CimInstance Win32_ComputerSystem | Select-Object TotalPhysicalMemory (Get-
CimInstance Win32_OperatingSystem).Caption
```

Mac (Terminal):

```
python3 --version sysctl -n machdep.cpu.brand_string sysctl -n hw.memsize sw_vers
-productVersion
```

4. Recording the Demo Video

Requirement: a 3-5 minute video showing one full run of the engine, narrated, including at least one AI-vs-AI match.

4.1 Screen recording tools

- Windows: press Win + G to open Xbox Game Bar, click the record button (or Win + Alt + R to start/stop directly).
- Mac: press Cmd + Shift + 5, select "Record Entire Screen" or "Record Selected Portion", click Record.

4.2 What to show and say

- Open the terminal in the src folder and run: python main.py demo
- While it runs, narrate: what Connect Four is, that this shows Rule-Based (X) vs Minimax (O), and point out the printed decision time next to each move.
- Mention that Minimax takes noticeably longer per move than Rule-Based because it searches 4 moves ahead using alpha-beta pruning, while Rule-Based just applies fixed priority rules instantly.
- When the game ends, state the result and briefly mention that this is one of 30 games run per pairing in the actual experiments, which is what the report's tables are based on.
- Optionally also show: python main.py human, and play a move or two yourself against the Minimax agent, to demonstrate interactivity.

4.3 After recording

Upload the video to YouTube (set visibility to "Unlisted", not private, so it's viewable via link) or to the shared Google Drive folder with link-sharing turned on. Copy the link — it goes in the README.md under "Demonstration Video".

5. Saving Your Changes Back to GitHub

If you edit anything (code, README, etc.), save it back so the rest of the team sees it. From inside the cp468-a2-connect4 folder:

```
git add . git commit -m "describe what you changed here" git push
```

If you generated new results.json / results.csv by running experiment.py, make sure to include them:

```
git add src/results.json src/results.csv git commit -m "Add experiment results from my machine" git push
```

5.1 Getting everyone else's latest changes

Before you start working, and anytime you want the newest version, run this from inside the cp468-a2-connect4 folder:

```
git pull
```

Do this every time before you start editing, to avoid conflicts with what teammates already pushed.

If git push is rejected because someone else pushed first, run "git pull" then try "git push" again. If you get a merge conflict (rare for this project since everyone is mostly touching different files), ask Asad for help resolving it before force-pushing anything.

6. Suggested Task Split

Person	Task
Daniyal Naqvi	Sections 3.3-3.4: run experiment.py, record specs, share results.csv/json with group
Omar Hamza	Section 4: record + narrate the demo, upload, share the link
Saad Siddiqui	Section 7: update Report.pdf and README with final results/specs/link, push final version, submit
Adam Ali	Read this doc + skim engine.py / agents.py so you can answer questions if asked, and confirm the contributions section in Report.pdf reflects what you actually did
Asad Malik	Already complete: engine, all three agents, experiment harness, initial report draft, GitHub repo setup. Available to help troubleshoot if anyone gets stuck.

7. Final Submission Checklist

- results.csv / results.json generated from a real run (not placeholder data)
- Report.pdf: Section 4 (Experimental Results) matches the real results; Section 4.1 has the real machine specs; Section 6 (Team Contributions) reflects what each person actually did
- README.md: video link filled in under "Demonstration Video"
- Video uploaded and link is viewable (test the link in an incognito/private window to confirm)
- Everything zipped into a single A2.zip: README.md, Report.pdf, and src/ (containing all .py files plus results.json and results.csv)
- Submitted as A2.zip per the assignment instructions

8. Questions?

If anything in the code doesn't make sense, engine.py, agents.py, experiment.py, and main.py are all commented and each function has a docstring explaining what it does. The Report.pdf also explains every agent's logic and the heuristic in detail — that's the fastest way to get up to speed on how the Minimax agent and heuristic work if you weren't the one who built them.